AI ASSISTANT CODING ASSIGNMENT -16.2

NAME : MOGILI KEERTHI

ROLL NO : 2303A51427

BATCH NO : 21

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Lab Objectives:

• Understand schema design principles for relational databases.

• Use AI-assisted tools to generate SQL schema definitions.

• Write and refine SQL queries (CRUD + aggregate operations).

• Explore how AI can optimize queries and ensure normalization.

Task Descrption-1: (Design Database Schema for Student Management System)

Instructions:

• Using AI tools, generate CREATE TABLE statements for a Student Management Database.

• Tables: Students, Courses, Enrollments.

• Include primary keys, foreign keys, and suitable data types.

Ensure the schema follows normalization rules (1NF, 2NF, 3NF).

Sample Code:

```
CREATE TABLE Students (
student_id INT PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(100)
);
CREATE TABLE Courses (
course_id INT PRIMARY KEY,
```

course_name VARCHAR(100),

credits INT

);

CREATE TABLE Enrollments (

enrollment_id INT PRIMARY KEY,

student_id INT,

course_id INT,

FOREIGN KEY (student_id) REFERENCES Students(student_id),

FOREIGN KEY (course_id) REFERENCES Courses(course_id)

);

Expected Output:

Tables Students, Courses, and Enrollments are created successfully.

Foreign key constraints are properly enforced.

Code:

```sql
-- SQLite
--Task Descrption-1: (Design Database Schema for Student Management System)
--Instructions:
-- Using AI tools, generate CREATE TABLE statements for a Student Management Database.
-- Tables: Students, Courses, Enrollments.
-- Include primary keys, foreign keys, and suitable data types.
--Ensure the schema follows normalization rules (1NF, 2NF, 3NF).
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    date_of_birth DATE NOT NULL
);
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    course_code TEXT UNIQUE NOT NULL,
    credits INTEGER NOT NULL
);
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    enrollment_date DATE NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)
);
```

Output:

```sql
CREATE TABLE Students (
student_id INTEGER PRIMARY KEY AUTOINCREMENT,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
date_of_birth DATE NOT NULL
);
```

```sql
CREATE TABLE Courses (
course_id INTEGER PRIMARY KEY AUTOINCREMENT,
course_name TEXT NOT NULL,
course_code TEXT UNIQUE NOT NULL,
credits INTEGER NOT NULL
);
```

```sql
CREATE TABLE Enrollments (
enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER NOT NULL,
course_id INTEGER NOT NULL,
enrollment_date DATE NOT NULL,
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id),
UNIQUE(student_id, course_id)
);
```

Observation:

The AI assistant simplifies the process of creating database tables by automatically generating clear and well-structured SQL code. Instead of writing everything manually, it provides ready-to-use CREATE TABLE statements that include primary keys, foreign keys, and necessary constraints. This approach saves time, minimizes errors, and demonstrates how AI can make complex tasks like designing a normalized database schema much easier.

Task Description- 2: (Insert Sample Records)

Instructions:

• Use AI tools to generate INSERT statements to populate the tables.

• Insert at least 3 students, 4 courses, and 5 enrollment records.

Verify data using SELECT queries.

Sample Code:

INSERT INTO Students VALUES

(1, 'Arjun', 'arjun@gmail.com'),

(2, 'Meera', 'meera@gmail.com'),

(3, 'Rahul', 'rahul@gmail.com');

INSERT INTO Courses VALUES

(101, 'Database Systems', 4),

(102, 'Data Structures', 3),

(103, 'Operating Systems', 4),

(104, 'AI Fundamentals', 3);

INSERT INTO Enrollments VALUES

(1, 1, 101),

(2, 1, 102),

(3, 2, 103),

(4, 3, 101),

(5, 3, 104);

Expected Output:

• Sample records are inserted successfully.

• FROM Students; displays all student records.

• SELECT * FROM Enrollments; displays all enrollments.

Code:

```sql
/*Task Description- 2: (Insert Sample Records)
Instructions:
• Use AI tools to generate INSERT statements to populate the
tables.
• Insert at least 3 students, 4 courses, and 5 enrollment records.
Verify data using SELECT queries.
*/
INSERT INTO Students (first_name, last_name, email, date_of_birth) VALUES
('John', 'Doe', 'john.doe@example.com', '2000-01-01'),
('Jane', 'Smith', 'jane.smith@example.com', '2001-05-15'),
('Emily', 'Johnson', 'emily.johnson@example.com', '2002-08-20');
INSERT INTO Courses (course_name, course_code, credits) VALUES
('Introduction to Computer Science', 'CS101', 3),
('Data Structures', 'CS102', 4),
('Database Systems', 'CS103', 3),
('Operating Systems', 'CS104', 4);
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES
(1, 1, '2023-09-01'),
(1, 2, '2023-09-01'),
(2, 1, '2023-09-01'),
(2, 3, '2023-09-01'),
(3, 4, '2023-09-01');
SELECT * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Enrollments;
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO Students (first_name, last_name, email, date_of_birth)
VALUES
('John', 'Doe', 'john.doe@example.com', '2000-01-01'),
('Jane', 'Smith', 'jane.smith@example.com', '2001-05-15'),
('Emily', 'Johnson', 'emily.johnson@example.com', '2002-08-20');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO Courses (course_name, course_code, credits) VALUES
('Introduction to Computer Science', 'CS101', 3),
('Data Structures', 'CS102', 4),
('Database Systems', 'CS103', 3),
('Operating Systems', 'CS104', 4);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES
(1, 1, '2023-09-01'),
(1, 2, '2023-09-01'),
(2, 1, '2023-09-01'),
(2, 3, '2023-09-01'),
(3, 4, '2023-09-01');
```

Output:

SQL ▾ 1 / 1 1 - 3 of 3

```
SELECT * FROM Students;
```

student_id	first_name	last_name	email	date_of_birth
1	John	Doe	john.doe@example.com	2000-01-01
2	Jane	Smith	jane.smith@example.com	2001-05-15
3	Emily	Johnson	emily.johnson@example.com	2002-08-20

SQL ▾ 1 / 1 1 - 4 of 4

```
SELECT * FROM Courses;
```

course_id	course_name	course_code	credits
1	Introduction to Computer Science	CS101	3
2	Data Structures	CS102	4
3	Database Systems	CS103	3
4	Operating Systems	CS104	4

SQL ▾ 1 / 1 1 - 5 of 5

```
SELECT * FROM Enrollments;
```

enrollment_id	student_id	course_id	enrollment_date
1	1	1	2023-09-01
2	1	2	2023-09-01
3	2	1	2023-09-01
4	2	3	2023-09-01
5	3	4	2023-09-01

Observation:

The AI assistant makes inserting sample records much easier and more efficient. Instead of manually creating values and writing multiple INSERT statements, it generates complete, ready-to-use SQL code with realistic student names, course details, and enrollment data. This saves time, reduces errors, and provides clear examples that help in understanding how the database functions in real-world scenarios.

Task Description- 3: (Perform CRUD Operations)

Instructions:

Use AI to generate SQL queries for basic CRUD operations:

• Retrieve all courses

• Update a student’s email

• Delete an enrollment record

• Validate the changes using SELECT statements.

Sample Code:

SELECT * FROM Courses;

UPDATE Students

SET email = 'arjun_new@gmail.com'

WHERE student_id = 1;

DELETE FROM Enrollments

WHERE enrollment_id = 5;

Expected Output:

• Updated email is reflected in Students table.

• Deleted enrollment record is removed successfully.

• Remaining data remains unchanged.

Code:

```
/*Task Description- 3: (Perform CRUD Operations)
Instructions:
Use AI to generate SQL queries for basic CRUD operations:
• Retrieve all courses
• Update a student’s email
• Delete an enrollment record
• Validate the changes using SELECT statements.
*/
-- Retrieve all courses
SELECT * FROM Courses;
-- Update a student’s email
UPDATE Students SET email = 'john.newemail@example.com' WHERE student_id = 1;
-- Delete an enrollment record
DELETE FROM Enrollments WHERE enrollment_id = 1;
-- Validate the changes using SELECT statements
SELECT * FROM Students;
SELECT * FROM Enrollments;
```

Output:

SQL ▾ 1 / 1 1 - 4 of 4

```
SELECT * FROM Courses;
```

course_id	course_name	course_code	credits
1	Introduction to Computer Science	CS101	3
2	Data Structures	CS102	4
3	Database Systems	CS103	3
4	Operating Systems	CS104	4

SQL ▾ 1 / 1 1 - 0 of 0

```
UPDATE Students SET email = 'john.newemail@example.com' WHERE student_id
= 1;
```

SQL ▾ 1 / 1 1 - 0 of 0

```
DELETE FROM Enrollments WHERE enrollment_id = 1;
```

SQL ▾ 1 / 1 1 - 3 of 3

```
SELECT * FROM Students;
```

student_id	first_name	last_name	email	date_of_birth
1	John	Doe	john.newemail@example.com	2000-01-01
2	Jane	Smith	jane.smith@example.com	2001-05-15
3	Emily	Johnson	emily.johnson@example.com	2002-08-20

SQL ▾ 1 / 1 1 - 4 of 4

```
SELECT * FROM Enrollments;
```

enrollment_id	student_id	course_id	enrollment_date
2	1	2	2023-09-01
3	2	1	2023-09-01
4	2	3	2023-09-01
5	3	4	2023-09-01

Observation:

The AI assistant simplifies performing CRUD operations by generating clear and well-structured SQL queries for each task. Instead of manually figuring out the syntax, it provides ready-to-use statements for retrieving data, updating records, and deleting entries, along with validation queries to verify changes. This saves time,

minimizes errors, and demonstrates how AI can make database management more efficient and practical.

Task Description- 4: (Execute Aggregate Queries)

Instructions:

Use AI tools to write aggregate queries:

• Count number of students enrolled per course

• Display courses with more than one student enrolled

Analyze the correctness of results.

Sample Code:

```
SELECT c.course_name, COUNT(e.student_id) AS student_count

FROM Courses c

JOIN Enrollments e ON c.course_id = e.course_id

GROUP BY c.course_name;

SELECT c.course_name

FROM Courses c

JOIN Enrollments e ON c.course_id = e.course_id

GROUP BY c.course_name

HAVING COUNT(e.student_id) > 1;
```

Expected Output:

• Correct count of students per course is displayed.

• Courses with multiple enrollments are listed accurately.

Code:

```sql
/*Task Description- 4: (Execute Aggregate Queries)
Instructions:
Use AI tools to write aggregate queries:
• Count number of students enrolled per course
• Display courses with more than one student enrolled
Analyze the correctness of results.
*/

-- Count number of students enrolled per course
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id;
-- Display courses with more than one student enrolled
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id
HAVING student_count > 1;

--explain why the result is empty for the second query
/*The result is empty for the second query because there are no courses that have more than one
student enrolled. Each course in the Enrollments table has only one student enrolled, so the
HAVING  clause filters out all courses, resulting in an empty result set.*/
```

Output:

SQL ▾ 1 / 1 1 - 4 of 4

```sql
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id;
```

course_id	student_count
1	1
2	1
3	1
4	1

SQL ▾ 1 / 1 1 - 0 of 0

```sql
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id
HAVING student_count > 1;
```

Observation:

The AI assistant simplifies writing aggregate queries by automatically generating accurate SQL syntax for tasks such as counting students per course and filtering courses with multiple enrollments. It also clearly explains the results—for instance,

why a query may return empty—demonstrating that it not only provides code but also enhances understanding of the underlying logic, making learning more intuitive.

Task Description- 5: (Query Optimization)

Instructions:

• Use AI tools to analyze query performance.

• Rewrite inefficient sub queries using JOINs and test both queries

for correctness.

Sample Code:

```
-- Original Query
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101
);
-- Optimized Query
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

Expected Output:

• Both queries return the same set of students.

• Optimized query performs better on large datasets.

This lab allows students to use AI tools like Copilot, Cursor, or

Gemini to:

• Generate schema definitions

• Auto-complete insert statements

• Write CRUD and aggregate SQL queries

• Optimize queries and ensure proper normalization

Code:

```sql
/*Task Description- 5: (Query Optimization)
Instructions:
• Use AI tools to analyze query performance.
• Rewrite inefficient sub queries using JOINs and test both queries
for correctness.*/
-- Inefficient subquery to retrieve student names enrolled in a specific course
SELECT first_name, last_name
FROM Students
WHERE student_id IN (SELECT student_id FROM Enrollments WHERE course_id = 1);
-- Optimized query using JOINs
SELECT s.first_name, s.last_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 1;
```

Output:

SQL ▾ 1 / 1 1 - 1 of 1

```sql
SELECT first_name, last_name
FROM Students
WHERE student_id IN (SELECT student_id FROM Enrollments WHERE course_id
= 1);
```

first_name	last_name
Jane	Smith

SQL ▾ 1 / 1 1 - 1 of 1

```sql
SELECT s.first_name, s.last_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 1;
```

first_name	last_name
Jane	Smith

Observation:

The AI assistant helps improve query performance by analyzing existing code and suggesting optimized alternatives. For example, it can replace an inefficient subquery with a JOIN, making the query faster and easier to understand. This demonstrates that the AI not only generates functional SQL but also promotes best practices, enhancing efficiency in real-world database operations.